

➤ **Question # 01:**

```
Class CircularQueue {
```

```
    Int[] queue = new int[5];
```

```
    Int front = -1;
```

```
    Int rear = -1;
```

```
    Void enqueue(int data) {
```

```
        If ((rear + 1) % 5 == front) {
```

```
            System.out.println("Queue is Full");
```

```
        } else {
```

```
            If (front == -1) {
```

```
                Front = 0;
```

```
            }
```

```
            Rear = (rear + 1) % 5;
```

```
            Queue[rear] = data;
```

```
        }
```

```
    }
```

```
    Void dequeue() {
```

```
        If (front == -1) {
```

```
            System.out.println("Queue is Empty");
```

```
        } else {
```

```
            System.out.println("Deleted: " + queue[front]);
```

```
            If (front == rear) {
```

```
        Front = -1;
        Rear = -1;
    } else {
        Front = (front + 1) % 5;
    }
}
}
```

```
Void display() {
```

```
    If (front == -1) {
        System.out.println("Queue is Empty");
        Return;
    }
```

```
    Int i = front;
```

```
    While (true) {
        System.out.println(queue[i]);
```

```
        If (i == rear) {
            Break;
        }
```

```
        I = (i + 1) % 5;
    }
}
```

```
Public static void main(String[] args) {
```

```
CircularQueue q = new CircularQueue();
```

```
q.enqueue(10);
```

```
q.enqueue(20);
```

```
q.enqueue(30);
```

```
q.enqueue(40);
```

```
System.out.println("Circular Queue:");
```

```
q.display();
```

```
q.dequeue();
```

```
q.dequeue();
```

```
q.enqueue(50);
```

```
q.enqueue(60);
```

```
System.out.println("After Operations:");
```

```
q.display();
```

```
}
```

```
}
```

➤ **Question # 02:**

```
Class CircularStack {
```

```
    Int[] stack = new int[5];
```

```
    Int top = -1;
```

```
    Void push(int data) {
```

```
        If (top == 4) {
```

```
    Top = -1;  
}
```

```
    Top++;  
    Stack[top] = data;  
}
```

```
Void pop() {
```

```
    If (top == -1) {  
        System.out.println("Stack is Empty");  
    } else {  
        System.out.println("Deleted: " + stack[top]);  
        Top--;  
    }  
}
```

```
Void display() {
```

```
    If (top == -1) {  
        System.out.println("Stack is Empty");  
    } else {  
        For (int i = top; i >= 0; i--) {  
            System.out.println(stack[i]);  
        }  
    }  
}
```

```
Public static void main(String[] args) {
```

```

CircularStack s = new CircularStack();

s.push(10);
s.push(20);
s.push(30);

System.out.println("Circular Stack:");
s.display();

s.pop();

System.out.println("After Pop:");
s.display();
}
}

```

➤ **Question # 03:**

```

Class Queue2DArray {

    Int[][] queue = new int[2][5];
    Int[] front = {0, 0};
    Int[] rear = {-1, -1};

    Void enqueue(int queueNo, int data) {
        If (rear[queueNo] == 4) {
            System.out.println("Queue is Full");
        } else {
            Rear[queueNo]++;
            Queue[queueNo][rear[queueNo]] = data;
        }
    }
}

```

```
    }  
}
```

```
Void dequeue(int queueNo) {  
    If (front[queueNo] > rear[queueNo]) {  
        System.out.println("Queue is Empty");  
    } else {  
        System.out.println("Deleted: “ +  
            Queue[queueNo][front[queueNo]]);  
        Front[queueNo]++;  
    }  
}
```

```
Void display(int queueNo) {  
    For (int i = front[queueNo]; i <= rear[queueNo]; i++) {  
        System.out.println(queue[queueNo][i]);  
    }  
}
```

```
Public static void main(String[] args) {  
    Queue2DArray q = new Queue2DArray();  
  
    q.enqueue(0, 10);  
    q.enqueue(0, 20);  
    q.enqueue(0, 30);  
  
    System.out.println("Queue:");  
    q.display(0);  
  
    q.dequeue(0);
```

```
}  
}
```

➤ **Question # 04:**

```
Class QueueLinkedList {  
    Node front, rear;  
    Class Node {  
        Int data;  
        Node next;  
        Node(int data) {  
            This.data = data;  
            This.next = null;  
        }  
    }  
    Void enqueue(int data) {  
        Node newNode = new Node(data);  
  
        If (rear == null) {  
            Front = rear = newNode;  
        } else {  
            Rear.next = newNode;  
            Rear = newNode;  
        }  
    }  
    Void dequeue() {  
        If (front == null) {  
            System.out.println("Queue is Empty");  
        } else {  
            System.out.println("Deleted: " + front.data);  
        }  
    }  
}
```

```

        Front = front.next;

        If (front == null) {
            Rear = null;
        }
    }
}

Void display() {
    Node temp = front;

    While (temp != null) {
        System.out.println(temp.data);
        Temp = temp.next;
    }
}

Public static void main(String[] args) {
    QueueLinkedList q = new QueueLinkedList();
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    System.out.println("Queue:");
    q.display();

    q.dequeue();

    System.out.println("After Dequeue:");
    q.display();
}
}

```

➤ **Question # 05:**

```
Class Stack2DArray {
    Int[][] stack = new int[2][5];
    Int[] top = {-1, -1};

    Void push(int stackNo, int data) {
        If (top[stackNo] == 4) {
            System.out.println("Stack is Full");
        } else {
            Top[stackNo]++;
            Stack[stackNo][top[stackNo]] = data;
        }
    }

    Void pop(int stackNo) {
        If (top[stackNo] == -1) {
            System.out.println("Stack is Empty");
        } else {
            System.out.println("Deleted: " +
                Stack[stackNo][top[stackNo]]);
            Top[stackNo]--;
        }
    }

    Void display(int stackNo) {
        For (int i = top[stackNo]; i >= 0; i--) {
            System.out.println(stack[stackNo][i]);
        }
    }
}
```

```

Public static void main(String[] args) {
    Stack2DArray s = new Stack2DArray();

    s.push(0, 10);
    s.push(0, 20);
    s.push(0, 30);

    System.out.println("Stack:");
    s.display(0);

    s.pop(0);
}
}

```

➤ **Question # 06:**

```

Class StackLinkedList {
    Node top;
    Class Node {
        Int data;
        Node next;

        Node(int data) {
            This.data = data;
            This.next = null;
        }
    }

    Void push(int data) {
        Node newNode = new Node(data);
        newNode.next = top;
    }
}

```

```

        top = newNode;
    }
    Void pop() {
        If (top == null) {
            System.out.println("Stack is Empty");
        } else {
            System.out.println("Deleted: " + top.data);
            Top = top.next;
        }
    }
    Void display() {
        Node temp = top;

        While (temp != null) {
            System.out.println(temp.data);
            Temp = temp.next;
        }
    }
    Public static void main(String[] args) {
        StackLinkedList s = new StackLinkedList();

        s.push(10);
        s.push(20);
        s.push(30);

        System.out.println("Stack:");
        s.display();

        s.pop();
    }
}

```

```
        System.out.println("After Pop:");  
        s.display();  
    }  
}
```